Introducción 1
Descripción del entorno y herramienta utilizada 2
Identificación de Vulnerabilidades 2
Resumen de Ocurrencias por Nivel de Riesgo 2
Clasificación por Tipología de Vulnerabilidad 3
Explotación de Vulnerabilidades 3
Reflected XSS 3
Stored XSS 4
Multi-Step Stored XSS 5
Reflected XSS Behind JavaScript 6
Reflected SQL Injection 6
Stored SQL Injection 7
File Inclusion (LFI) 8
Directory Traversal 9
Weak Username/Password 10
SessionID Vulnerability 11
Parameter Manipulation (IDOR) 12
Logic Flaw 13
Medidas de prevención 14
Reflected XSS 14
Stored XSS 14
Multi-Step Stored XSS 14
Reflected XSS Behind JavaScript 15
Reflected SQL Injection 15
Stored SQL Injection 15
File Inclusion (LFI) 15
Directory Traversal 16
Weak Username/Password 16
SessionID Vulnerability 16
Parameter Manipulation (IDOR) 16
Logic Flaw 16
Conclusión 17

Introducción

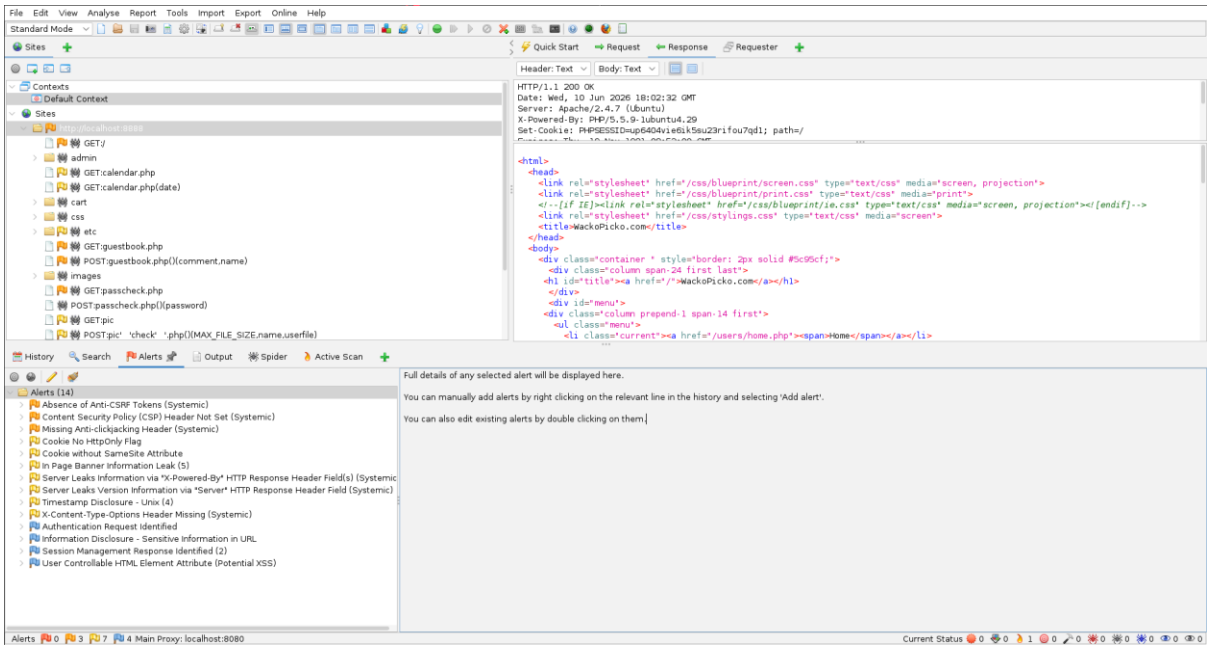
Descripción del entorno y herramienta utilizada

Para realizar este ejercicio se ha trabajado con WackoPicko, una aplicación web con vulnerabilidades para practicar técnicas de análisis y auditoría de seguridad. La aplicación la he en una máquina virtual y he accedido desde el navegador en http://localhost:8888.

Identificación de Vulnerabilidades

Para la identificación de vulnerabilidades se ha utilizado OWASP ZAP en modo ATTACK, configurado como proxy en localhost:8080, apuntando contra la aplicación WackoPicko accesible en http://localhost:8888. Se realizó un rastreo automático mediante el Spider y posteriormente un Active Scan sobre los dos sitios detectados (http://localhost:8888 y https://localhost:8888).

El escaneo ha identificado un total de 16 alertas.



Resumen de Ocurrencias por Nivel de Riesgo

El escaneo ha identificado un total de 16 alertas, distribuidas de la siguiente forma según su nivel de riesgo:

Nivel de Riesgo	Nº de Ocurrencias
Alto	1
Medio	4
Bajo	7
Informativo	4

TOTAL	**16**

Clasificación por Tipología de Vulnerabilidad

A continuación se muestra la clasificación completa de todas las alertas identificadas por OWASP ZAP, agrupadas por tipología:

Vulnerabilidad	**Nivel**	**Tipología**
Cross Site Scripting (DOM Based)	Alto	XSS
User Controllable HTML Element Attribute (Potential XSS)	Medio	XSS
Absence of Anti-CSRF Tokens	Medio	CSRF
Content Security Policy (CSP) Header Not Set	Medio	Configuración
Missing Anti-clickjacking Header	Medio	Configuración
HTTP Only Site	Bajo	Configuración
Cookie No HttpOnly Flag	Bajo	Gestión de Sesión
Cookie without SameSite Attribute	Bajo	Gestión de Sesión
In Page Banner Information Leak (5)	Bajo	Fuga de Información
Server Leaks Info via X-Powered-By Header	Bajo	Fuga de Información
Server Leaks Version via Server Header	Bajo	Fuga de Información
X-Content-Type-Options Header Missing	Bajo	Configuración
Timestamp Disclosure - Unix	Informativo	Fuga de Información
Authentication Request Identified	Informativo	Autenticación
Information Disclosure - Sensitive Info in URL	Informativo	Fuga de Información
Session Management Response Identified (2)	Informativo	Gestión de Sesión

Explotación de Vulnerabilidades

Reflected XSS

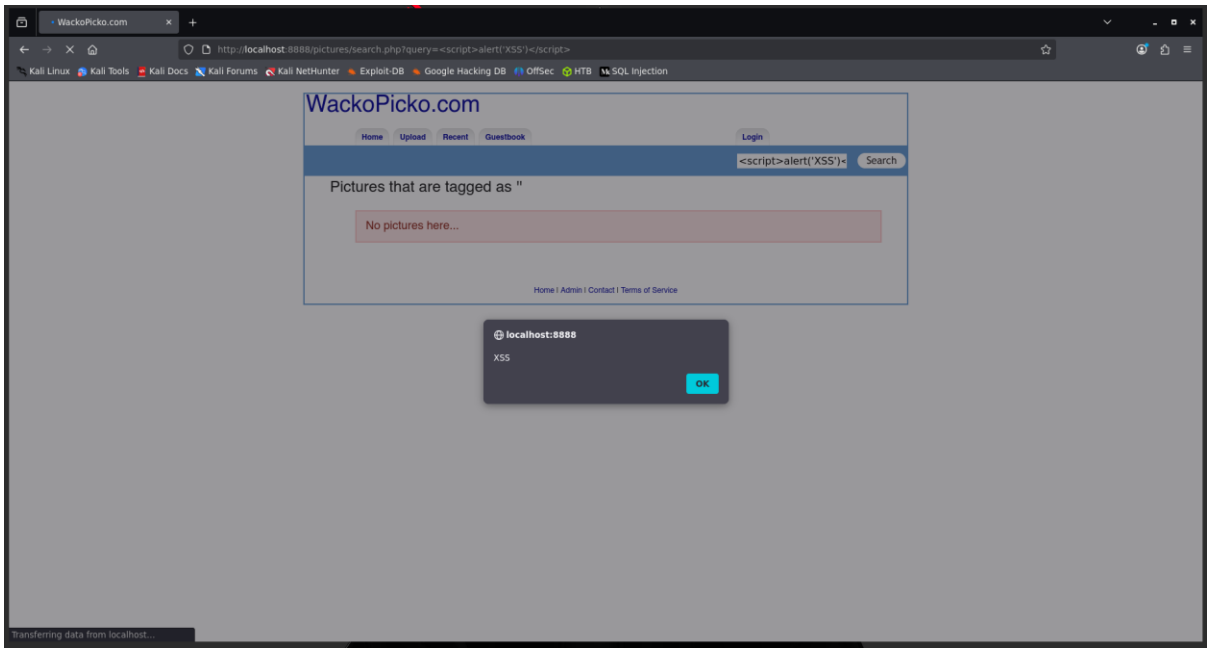
URL: http://localhost:8888/pictures/search.php

En la página de búsqueda de imágenes vi que el campo de búsqueda devuelve el texto introducido directamente en la página sin ningún tipo de filtro. Probé a meter código JavaScript directamente en el buscador para ver si se ejecutaba.

Introduje lo siguiente en el campo de búsqueda:

<script>alert('XSS')</script>

El navegador mostró la alerta, confirmando que se puede ejecutar código JavaScript. Esto permitiría a un atacante robar cookies de sesión o redirigir a la víctima a páginas maliciosas.



Stored XSS

URL: http://localhost:8888/guestbook.php

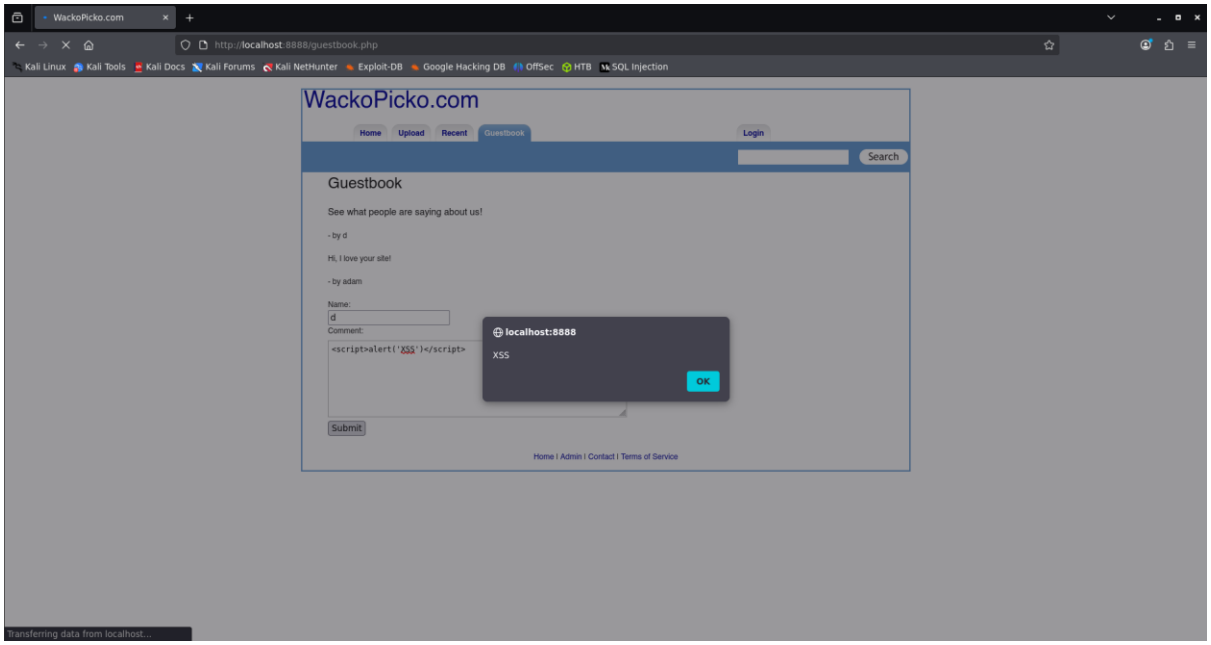
En esta url comprobé que los comentarios se almacenan en la base de datos sin ninguna validación. A diferencia del caso anterior, aquí el payload queda guardado de forma permanente y se ejecuta cada vez que alguien visita la página.

Introduje el siguiente payload en el campo de comentario:

<script>alert('XSS')</script>

Al enviar el formulario el comentario se guardó en la base de datos. Al recargar la página el script se ejecutó automáticamente sin necesidad de ninguna interacción adicional.

Esto es peligroso porque con un solo payload se puede afectar a todos los usuarios que visiten el guestbook, sin necesidad de engañar a nadie para que pulse ningún enlace.



Multi-Step Stored XSS

URL: http://localhost:8888/pictures/view.php?picid=3

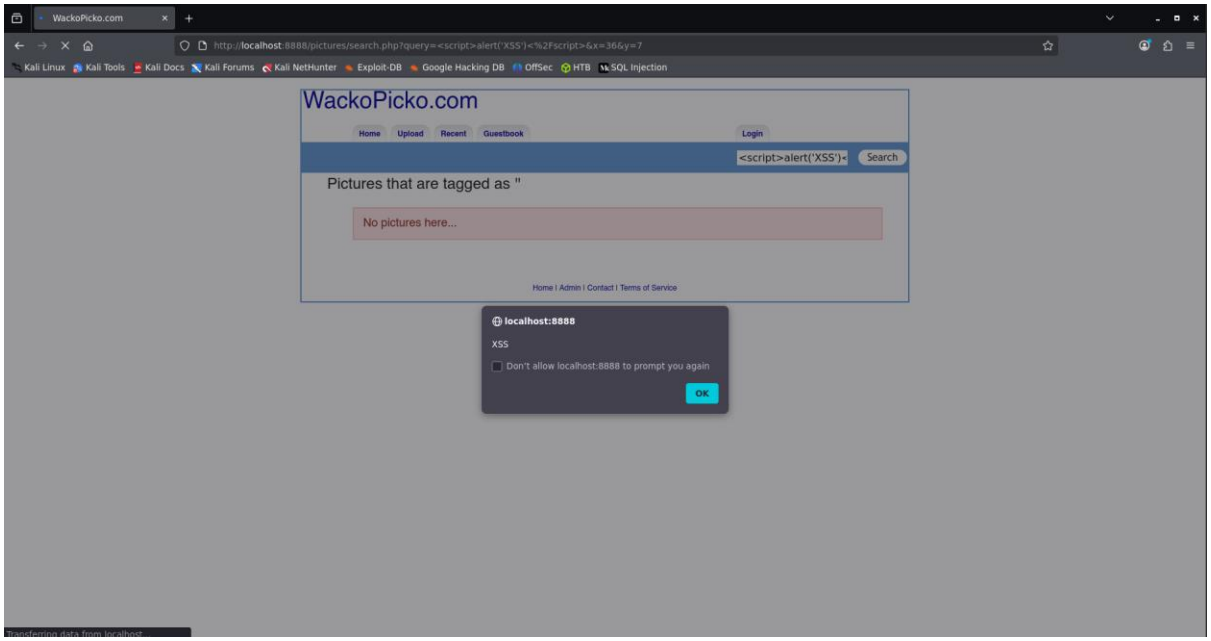
En la sección de comentarios de imágenes existe un paso previo de previsualización antes de publicar. Podría parecer que ahí se hace alguna validación, pero comprobé que no es así.

Introduje el siguiente payload en el campo de comentario de una imagen:

<script>alert('XSS')</script>

Al pulsar submit, el script ya se ejecutó en esa pantalla. Al confirmar y publicar, el payload quedó almacenado y se siguió ejecutando en cada visita a esa imagen.

El paso de previsualización no aplica ningún filtro, por lo que el resultado es idéntico al stored XSS anterior. La diferencia es que aquí la vulnerabilidad está más escondida detrás de un flujo de dos pasos que da una falsa sensación de control.



Reflected XSS Behind JavaScript

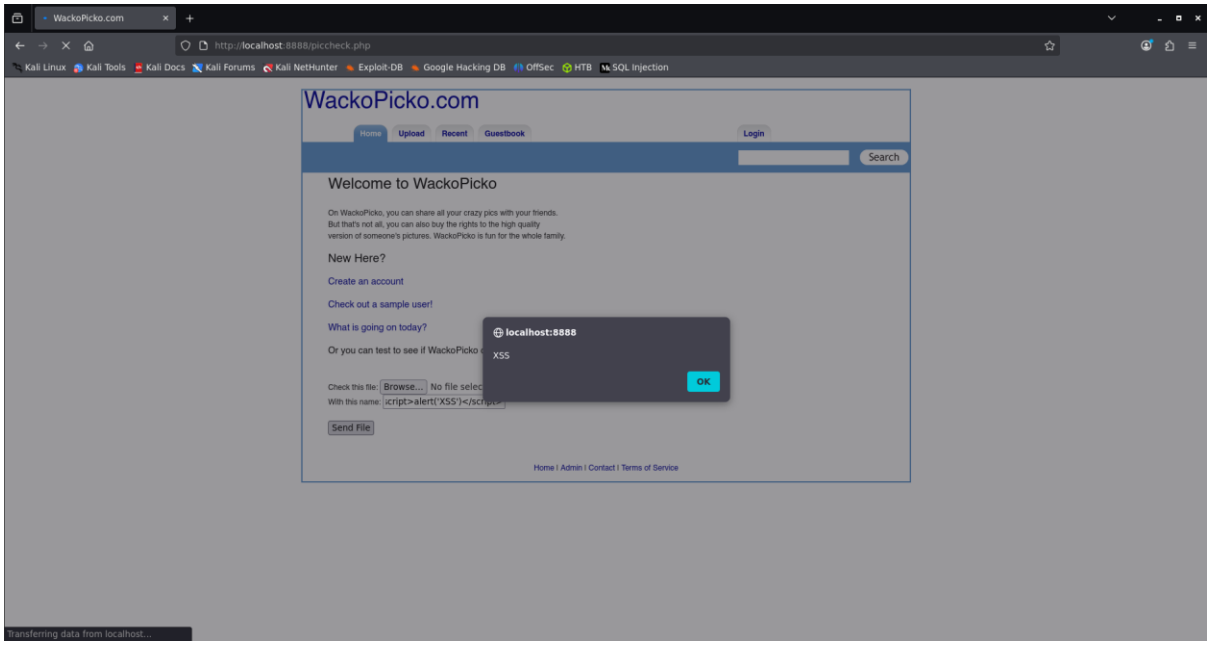
URL: http://localhost:8888/piccheck.php

En esta página 'name' pasa por una función JavaScript antes de mostrarse. Aun así, el valor se sigue insertando en el DOM sin sanitizar.

Introduje el siguiente payload en la URL:

http://localhost:8888/piccheck.php?name=<script>alert('XSS')</script>

El script se ejecutó sin problema. Pasar un valor por JavaScript no es suficiente para protegerse contra XSS si luego se inserta en el DOM sin sanitizar correctamente.



Reflected SQL Injection

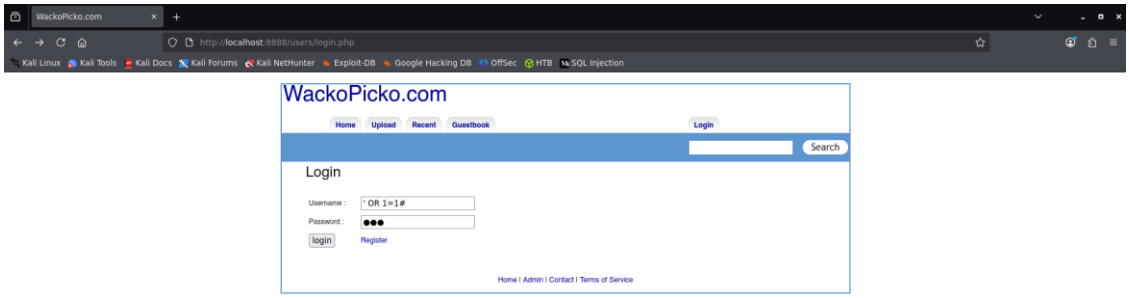
URL: http://localhost:8888/users/login.php

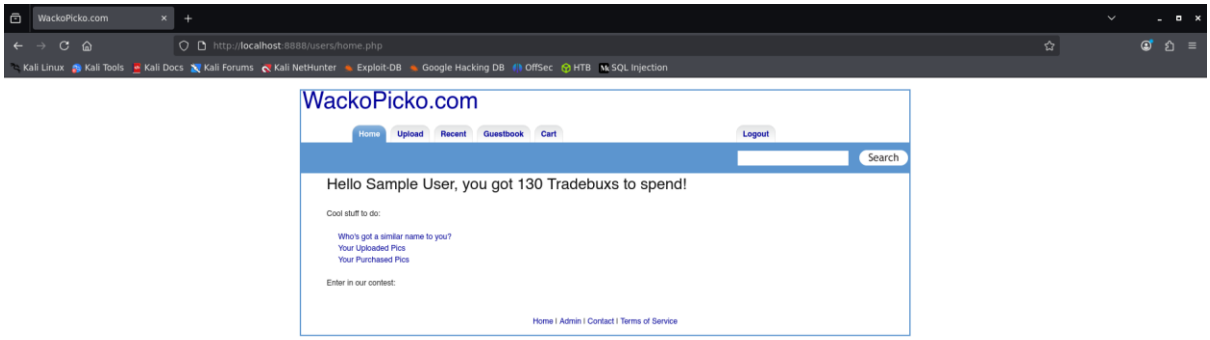
Probé el formulario de login con caracteres especiales típicos de SQLi. El campo username se mete directamente en la consulta SQL sin usar sentencias preparadas.

Payload introducido en el campo de usuario:

' OR 1=1#

La consulta siempre devuelve verdadero y la aplicación me dejó acceder sin tener credenciales válidas. Además, durante las pruebas la aplicación llegó a mostrar errores internos de MySQL, exponiendo rutas como /app/include/database.php y el usuario de base de datos www-data, lo que también es una fuga de información importante.





Stored SQL Injection

URL: http://localhost:8888/users/register.php a

http://localhost:8888/users/similar.php

El campo first name del registro se guarda en la base de datos sin ningún tipo de sanitización. Ese dato se utiliza después en /similar.php para otra consulta SQL, por lo que la inyección viaja de una página a otra.

Me registré con estos datos:

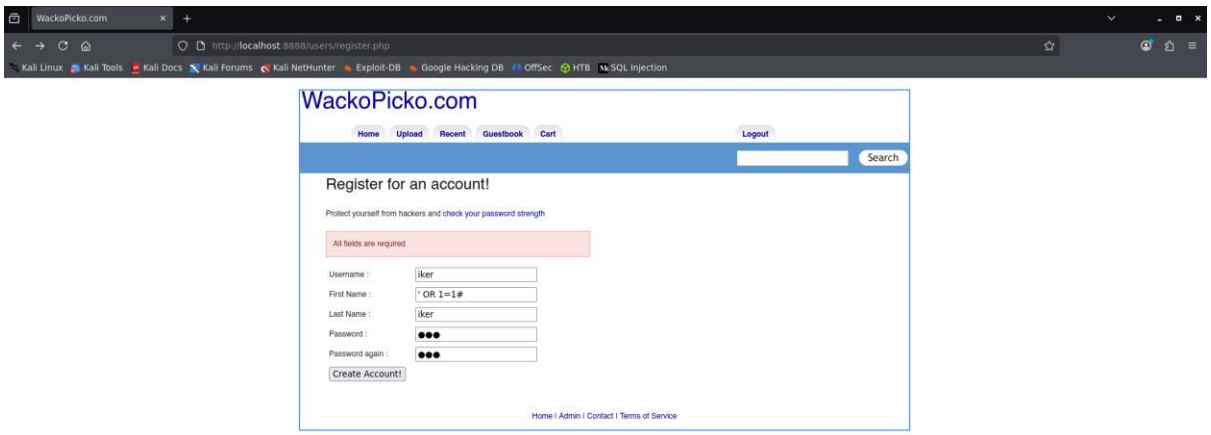
Username: iker

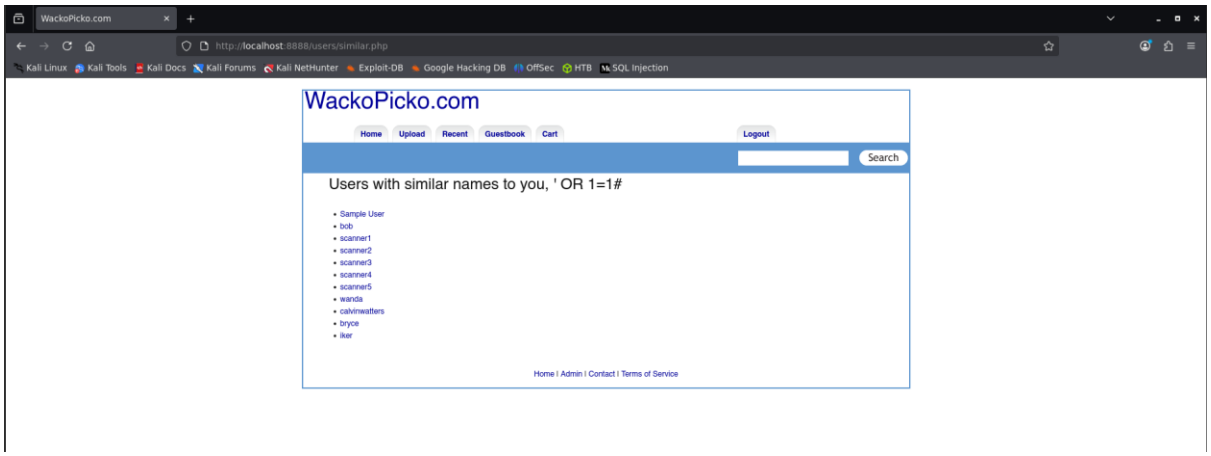
First Name: ' OR 1=1#

Last Name: test

Password: 1234

Al acceder a /similar.php con esa cuenta, la consulta que usa ese nombre devolvió todos los usuarios de la base de datos.

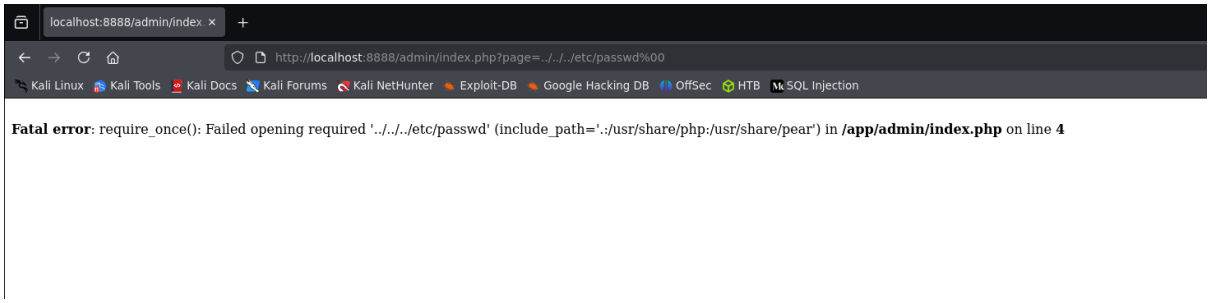




File Inclusion (LFI)

El panel de administración carga secciones usando ‘page’ sin validarlo. Intenté incluir el fichero /etc/passwd añadiendo %00 al final.

Aunque el fichero no se mostró por el uso de require_once(), el error confirma que el parámetro se usa directamente para incluir ficheros sin ninguna validación, y además expone la ruta interna /app/admin/index.php.



Directory Traversal

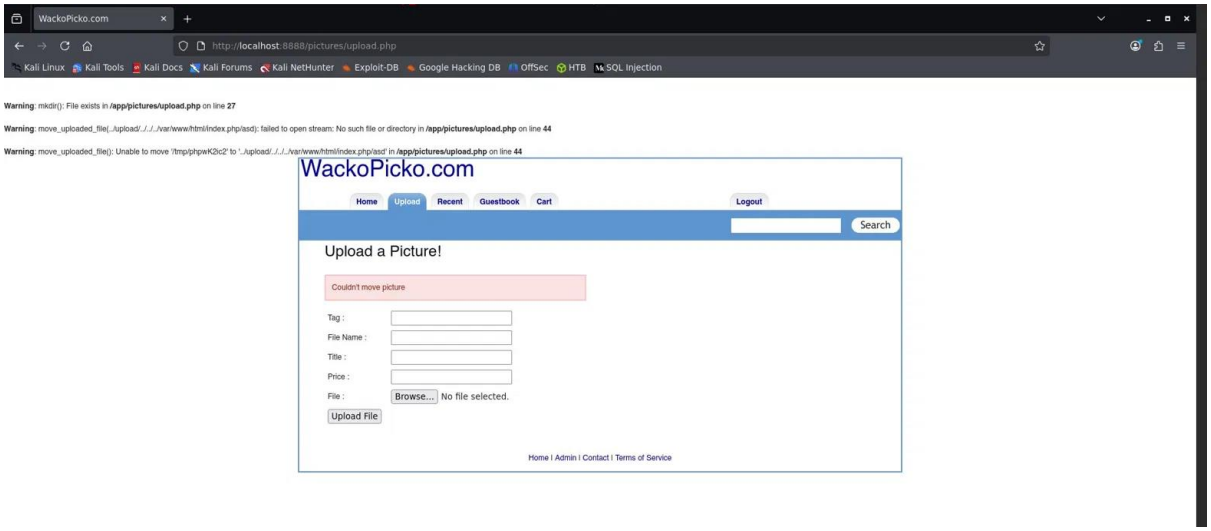
URL: http://localhost:8888/pictures/upload.php

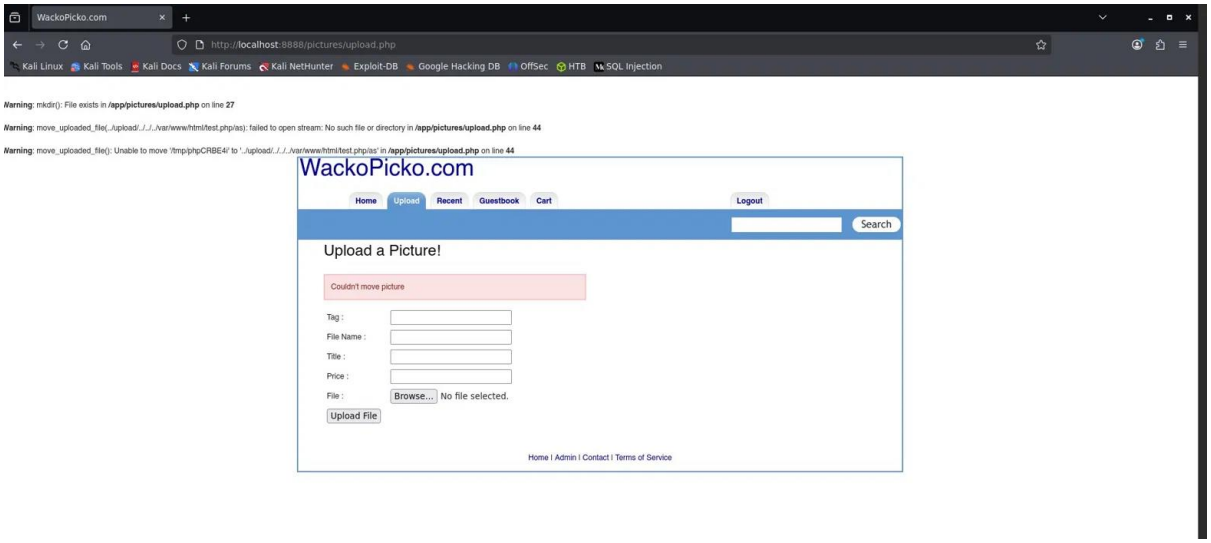
Al subir una imagen, el campo tag se usa para construir la ruta donde se guarda el fichero. No existe ninguna validación sobre ese campo, por lo que se puede usar para intentar escribir fuera del directorio de uploads.

Rellené el formulario con cualquier imagen y en el campo tag metí:

../../../var/www/html/test.php

El error confirma que la aplicación intentó mover el fichero a la ruta que le indiqué, fuera del directorio de uploads. Si el directorio tuviera permisos de escritura, el fichero se habría guardado allí y sería posible subir una webshell para comprometer el servidor.





Weak Username/Password

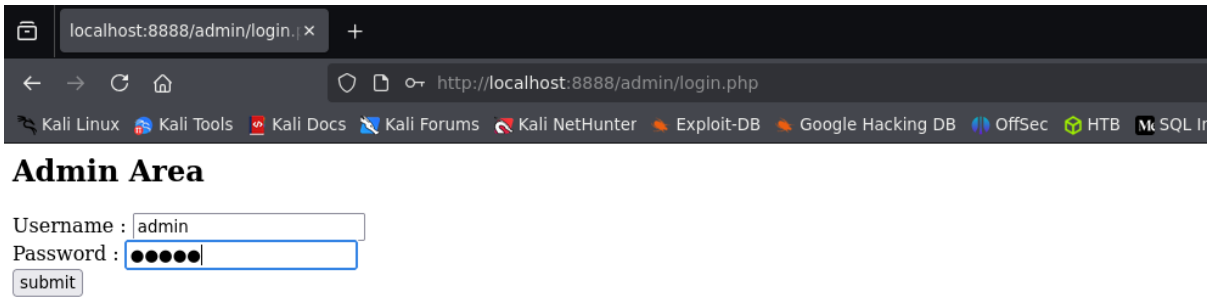
URL: http://localhost:8888/admin/login.php

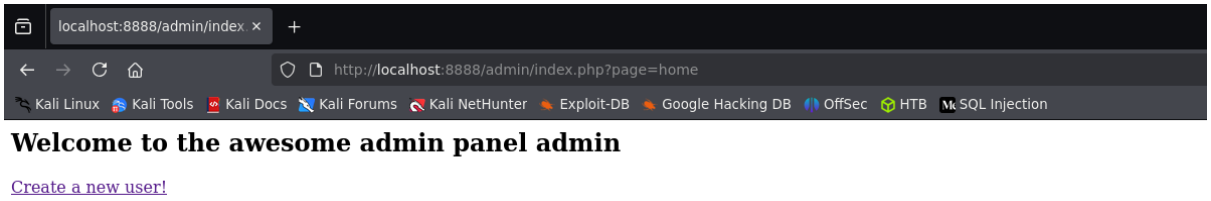
Al encontrar el panel de administración probé directamente las credenciales por defecto más comunes:

Usuario: admin

Contraseña: admin

Accedí al panel de administración con el control total de la aplicación. Este es el fallo más básico y el primero que cualquier atacante probaría al encontrar un panel de login.



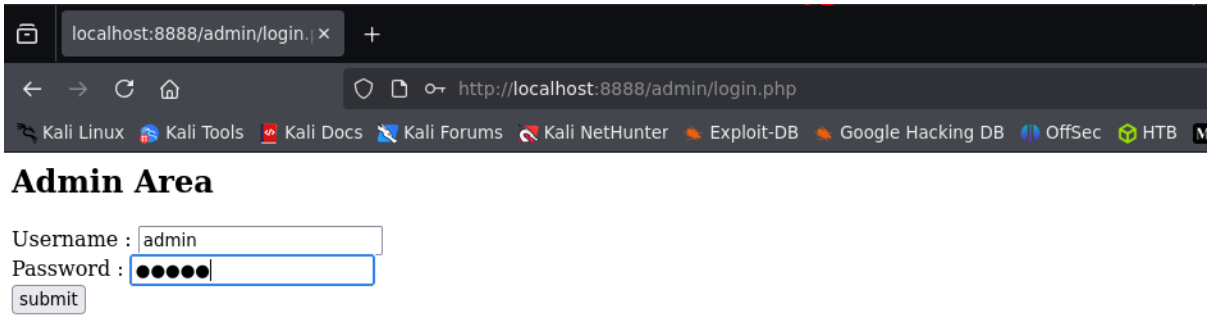


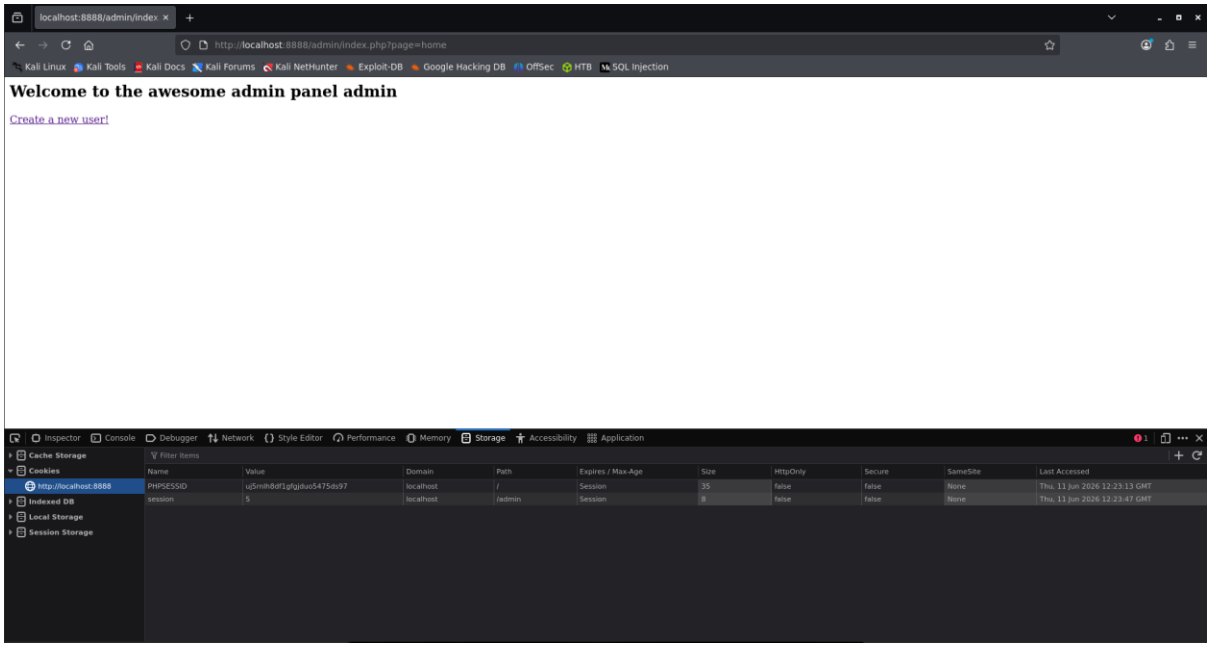
SessionID Vulnerability

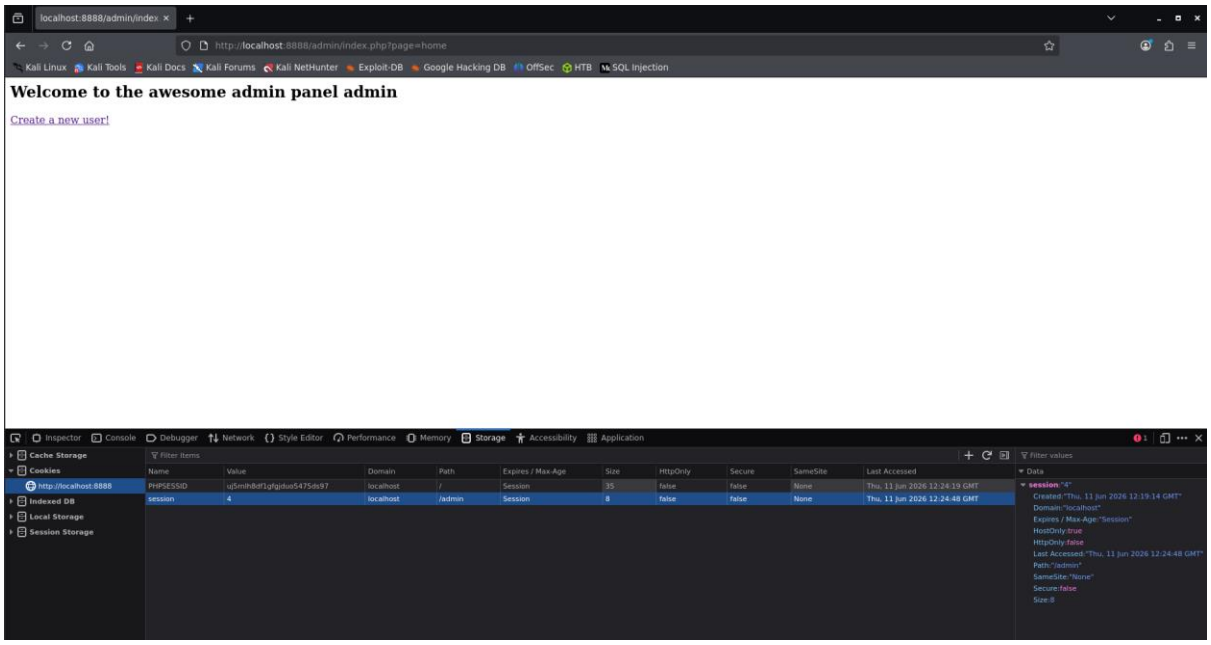
URL: http://localhost:8888/admin/login.php

Tras hacer login como administrador, abrí las herramientas de desarrollador del navegador para revisar las cookies de sesión.

Encontré una cookie llamada session con el valor numérico 4. Un atacante podría probar valores como 1, 2 o 3 para intentar acceder a sesiones activas de otros usuarios o administradores sin necesitar sus credenciales.







Parameter Manipulation (IDOR)

URL: http://localhost:8888/users/sample.php

La página de perfil de usuario recibe el identificador directamente por la URL. La aplicación no comprueba si el usuario autenticado tiene permiso para ver el perfil que está solicitando.

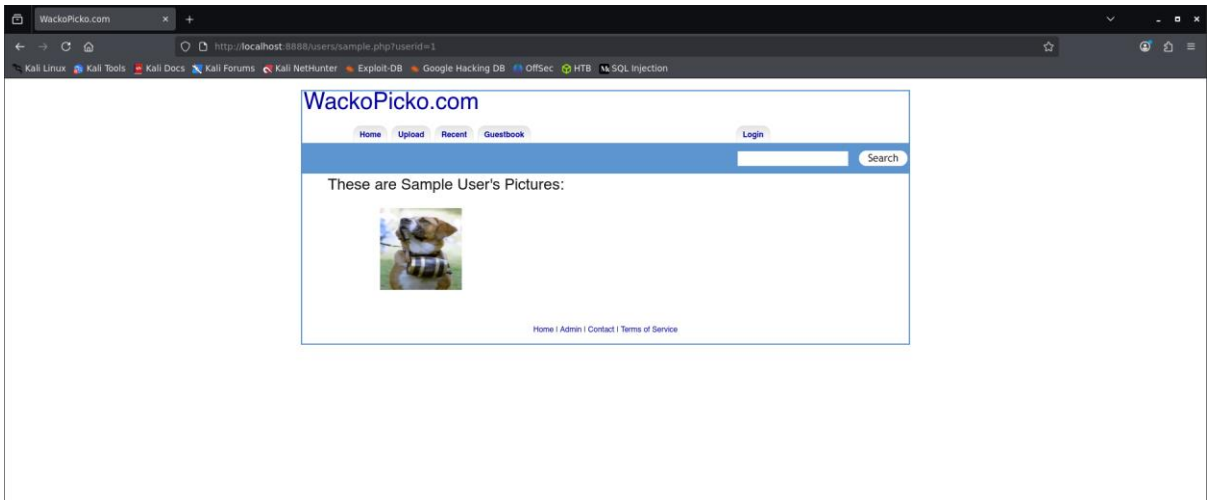
Sin estar autenticado, fui modificando el parámetro userid para ver distintos perfiles:

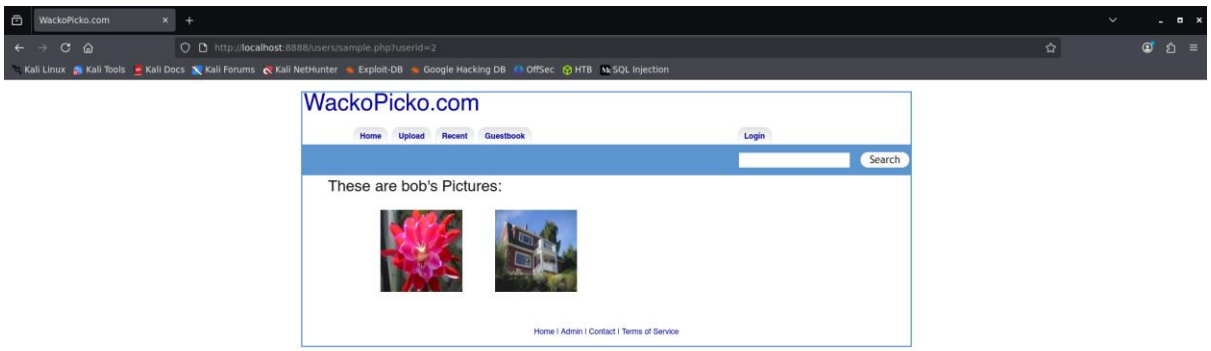
http://localhost:8888/users/sample.php?userid=1

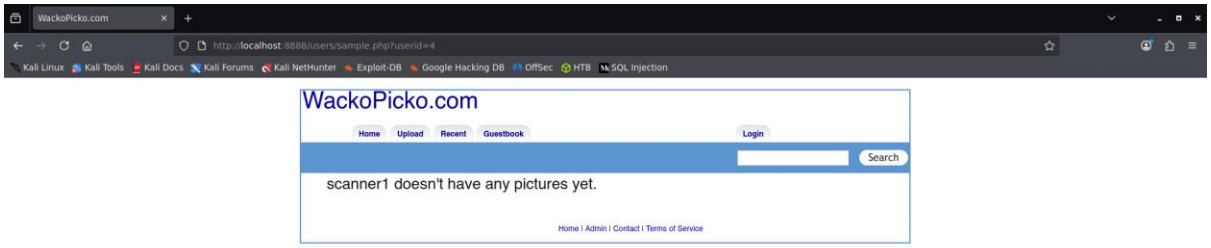
http://localhost:8888/users/sample.php?userid=2

http://localhost:8888/users/sample.php?userid=4

Pude ver las fotos y datos de distintos usuarios sin tener ninguna autorización para ello.





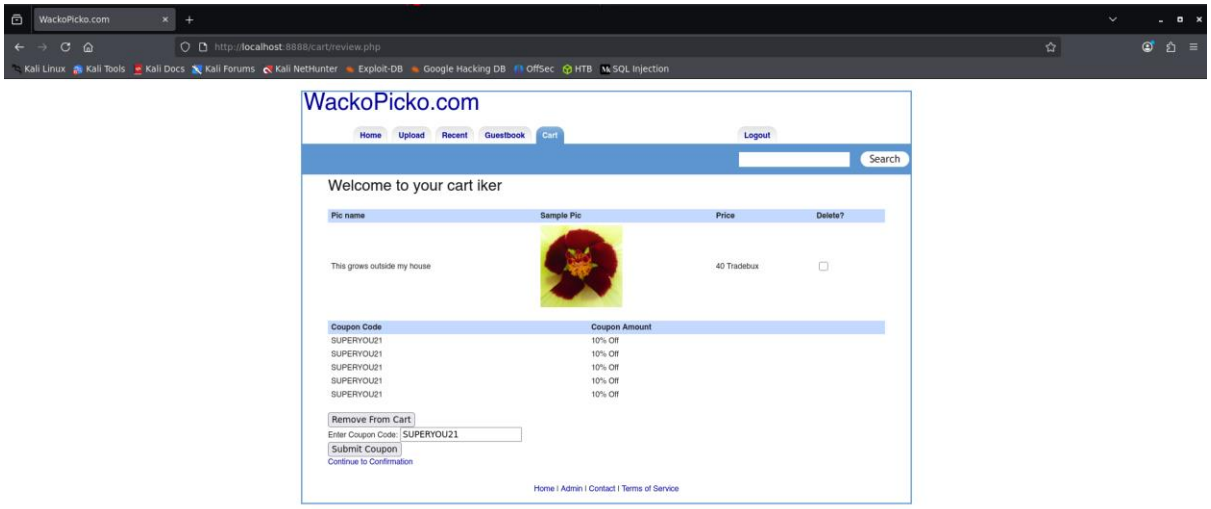


Logic Flaw

URL: http://localhost:8888/cart/review.php

Durante el proceso de compra probé a introducir el cupón de descuento SUPERYOU21 varias veces seguidas en el mismo pedido. La aplicación no lleva ningún control de cuántas veces se ha aplicado un cupón.

Añadí una imagen al carrito y fui aplicando el cupón SUPERYOU21 repetidamente hasta que el precio total llegó a 0.



Medidas de prevención

Reflected XSS

La causa del problema está en que la aplicación devuelve el valor del parámetro query directamente en el HTML. La solución es usar htmlspecialchars() antes de mostrar cualquier dato que venga del usuario. EL código corregido sería así:

echo htmlspecialchars($_GET['query'], ENT_QUOTES, 'UTF-8');

Stored XSS

El comentario se guarda y se muestra sin ningún filtro. Hay que sanitizar al guardar y al mostrar:

- Al guardar:

$comment = htmlspecialchars($_POST['comment'], ENT_QUOTES, 'UTF-8');

- Al mostrar:

echo htmlspecialchars($row['comment'], ENT_QUOTES, 'UTF-8');

Multi-Step Stored XSS

La corrección es la misma que en el Stored XSS. En el paso de previsualización se debe aplicar el mismo htmlspecialchars() que se aplica al guardar, de forma que ninguno permita inyectar código.

Reflected XSS Behind JavaScript

Si se usa JavaScript para insertar el valor en la página, hay que usar textContent en lugar de innerHTML, que interpreta el HTML y ejecuta scripts. El código corregido sería:

```
document.getElementById('output').textContent = name;
```

Reflected SQL Injection

La consulta del login construye la query uniendo directamente el input del usuario. La solución es usar sentencias preparadas con PDO. El código corregido sería:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?");

$stmt->execute([$username, $password]);
```

Stored SQL Injection

Misma corrección. El campo first_name del registro tiene que usarse con sentencias preparadas, nunca uniéndose directamente:

```
$stmt = $pdo->prepare("INSERT INTO users (first_name) VALUES (?)");

$stmt->execute([$first_name]);
```

File Inclusion (LFI)

El parámetro page no debería usarse directamente para incluir ficheros. La solución es definir una lista blanca de páginas permitidas y rechazar cualquier valor que no esté en ella. El código corregido sería:

```
$allowed = ['login', 'dashboard', 'users'];

$page = $_GET['page'];

if (in_array($page, $allowed)) {

  require_once($page . '.php');
```

```
} else {
   require_once('login.php');
}
```

Directory Traversal

El campo tag debe sanitizarse eliminando cualquier secuencia de path traversal. La función basename() de PHP se queda solo con el nombre del fichero, descartando cualquier ruta. El código corregido sería:

```
$tag = basename($_POST['tag']);
move_uploaded_file($tmp, '../upload/' . $tag . '/' . $filename);
```

Weak Username/Password

Hay que cambiar las credenciales por defecto durante la instalación y no permitir que la aplicación funcione con admin/admin. Como medida adicional, implementar un límite de intentos fallidos de login para dificultar ataques de fuerza bruta.

SessionID Vulnerability

El identificador de sesión debe generarse de forma segura, nunca con un valor incremental. El código corregido sería:

```
$session_id = bin2hex(random_bytes(32));
```

Parameter Manipulation (IDOR)

La aplicación tiene que comprobar que el userid solicitado corresponde al usuario que está autenticado en la sesión:

```
if ($_GET['userid'] != $_SESSION['user_id']) {
   header('HTTP/1.1 403 Forbidden');
   exit('No tienes permiso para ver este perfil');
}
```

Logic Flaw

La aplicación debe registrar en base de datos si un cupón ya fue aplicado en un pedido y bloquearlo si se intenta usar de nuevo:

```
if (coupon_already_used($coupon_code, $order_id)) {
  echo 'Este cupón ya ha sido utilizado en este pedido';
} else {
  apply_coupon($coupon_code, $order_id);
  mark_coupon_as_used($coupon_code, $order_id);
}
```

Conclusión

Después de analizar WackoPicko tanto con OWASP ZAP como de forma manual, he podido identificar y explotar un total de 15 vulnerabilidades.

Las más críticas son la inyección de comandos, que daría acceso completo al servidor, la SQL Injection, que expone toda la base de datos, y la inclusión de ficheros, con la que se podrían leer ficheros sensibles del sistema. A esto se suma que la aplicación tenía las credenciales por defecto admin/admin activas, lo que facilita enormemente el acceso inicial.

También llama la atención que varias vulnerabilidades como el Logic Flaw o la SessionID Vulnerability no aparecieron en el escaneo de ZAP. Esto demuestra que las herramientas automáticas son un buen punto de partida, pero no sustituyen al análisis manual, especialmente cuando los fallos están en la lógica de negocio de la aplicación.

En general WackoPicko es un ejemplo claro de lo que no hay que hacer: inputs sin validar, consultas SQL construidas por concatenación, credenciales por defecto y ausencia de controles de acceso básicos. Todos estos problemas tienen solución aplicando buenas prácticas de desarrollo seguro desde el principio.